NBA Analysis Project: Model Benchmarks & 2027 Season Projections

Kailot C. Harris

2026-09-06

Introduction

This project develops an end-to-end NBA game outcome prediction pipeline and season simulation model using `hoopR` team box scores (2019–2026). Key enhancements in this updated analysis include:

1. **Elimination of Data Leakage:** Removing calendar `season` from predictor sets and capping offseason `days_rest` artifacts.
2. **Offseason Transition Comparisons:** Evaluating 3 distinct methods for handling rolling statistics across season boundaries (Fresh Reset, 50% Mean Regression, and Exponential Decay).
3. **Model Benchmarking:** Comparing Logistic Regression (baseline), Random Forest (`ranger`), and Gradient Boosted Trees (`xgboost`) across out-of-sample metrics (Accuracy, ROC-AUC, Brier Score, Log-Loss).
4. **Dynamic 2027 Season Simulation:** Running 10,000 Monte Carlo season simulations with dynamic 82-game schedule pacing.

Load packages:

```
library(tidyverse)
library(tidymodels)
library(hoopR)
library(slider)
library(xgboost)
library(ggridges)
```

Data Ingestion & Caching

Load the box score data from the `hoopR` API for seasons 2019 through 2026, using local caching to optimize execution:

```
cache_file <- "box_scores_cache.rds"

if (file.exists(cache_file)) {
  box_scores <- readRDS(cache_file)
} else {
  box_scores <- hoopR::load_nba_team_box(seasons = 2019:2026)
  saveRDS(box_scores, cache_file)
}

# Summary check
cat("Total team box score records:", nrow(box_scores), "\n")
```

```
## Total team box score records: 20480
```

Feature Engineering & Offseason Transition Experiments

We compute team rolling averages (18-game window) for key performance indicators (eFG%, Turnover Rate, Free Throw Rate, True Shooting %, Points, Rebounds, Assists, Steals, Blocks).

To resolve offseason transition artifacts, we test three methodologies: - **Method A (Fresh Reset):** Rolling windows reset cleanly at game 1 of each new season (`group_by(season, team_id)`). - **Method B (50% Regression to Mean):** Transitioning to a new season regresses team baseline ratings 50% toward the league average. - **Method C (Exponential Decay Carryover):** Rolling stats carry over across seasons with an 80/20 decay blend.

```
n_rolling_days <- 18

build_feature_df <- function(data, window_size = n_rolling_days, method = "reset") {
  if (method == "reset") {
    features <- data |>
      arrange(team_id, season, game_date) |>
      group_by(season, team_id) |>
      mutate(
        efg = (field_goals_made + 0.5 * three_point_field_goals_made) / pmax(1, field_goals_attempted),
        tov = total_turnovers / pmax(1, field_goals_attempted + 0.44 * free_throws_attempted + total_tu
        ftr = free_throws_attempted / pmax(1, field_goals_attempted),
        ts  = team_score / pmax(1, 2 * (field_goals_attempted + 0.44 * free_throws_attempted)),

        roll_pts_scored = slide_dbl(team_score, mean, .before = window_size, .after = -1),
        roll_oreb       = slide_dbl(offensive_rebounds, mean, .before = window_size, .after = -1),
        roll_dreb       = slide_dbl(defensive_rebounds, mean, .before = window_size, .after = -1),
        roll_ast        = slide_dbl(assists, mean, .before = window_size, .after = -1),
        roll_stl        = slide_dbl(steals, mean, .before = window_size, .after = -1),
        roll_blk        = slide_dbl(blocks, mean, .before = window_size, .after = -1),
        roll_tov        = slide_dbl(turnovers, mean, .before = window_size, .after = -1),
        roll_efg        = slide_dbl(efg, mean, .before = window_size, .after = -1),
        roll_tovp       = slide_dbl(tov, mean, .before = window_size, .after = -1),
        roll_ftr        = slide_dbl(ftr, mean, .before = window_size, .after = -1),
        roll_ts         = slide_dbl(ts, mean, .before = window_size, .after = -1),
        days_rest       = pmin(7, coalesce(as.numeric(game_date - lag(game_date)), 3))
      ) |>
      ungroup()
  } else {
    features <- data |>
      arrange(team_id, game_date) |>
      group_by(team_id) |>
      mutate(
        efg = (field_goals_made + 0.5 * three_point_field_goals_made) / pmax(1, field_goals_attempted),
        tov = total_turnovers / pmax(1, field_goals_attempted + 0.44 * free_throws_attempted + total_tu
        ftr = free_throws_attempted / pmax(1, field_goals_attempted),
        ts  = team_score / pmax(1, 2 * (field_goals_attempted + 0.44 * free_throws_attempted)),

        roll_pts_scored = slide_dbl(team_score, mean, .before = window_size, .after = -1),
        roll_oreb       = slide_dbl(offensive_rebounds, mean, .before = window_size, .after = -1),
        roll_dreb       = slide_dbl(defensive_rebounds, mean, .before = window_size, .after = -1),
        roll_ast        = slide_dbl(assists, mean, .before = window_size, .after = -1),
        roll_stl        = slide_dbl(steals, mean, .before = window_size, .after = -1),
        roll_blk        = slide_dbl(blocks, mean, .before = window_size, .after = -1),
        roll_tov        = slide_dbl(turnovers, mean, .before = window_size, .after = -1),
```

```
      roll_efg        = slide_dbl(efg, mean, .before = window_size, .after = -1),
      roll_tovp       = slide_dbl(tov, mean, .before = window_size, .after = -1),
      roll_ftr        = slide_dbl(ftr, mean, .before = window_size, .after = -1),
      roll_ts         = slide_dbl(ts, mean, .before = window_size, .after = -1),
      is_new_season   = coalesce(season != lag(season), FALSE),
      days_rest       = pmin(7, coalesce(as.numeric(game_date - lag(game_date)), 3))
    )

  if (method == "mean_regress") {
    features <- features |>
      mutate(
        roll_pts_scored = if_else(is_new_season, 0.5 * roll_pts_scored + 0.5 * mean(team_score, na.rm
        roll_efg = if_else(is_new_season, 0.5 * roll_efg + 0.5 * mean(efg, na.rm = TRUE), roll_efg),
        roll_ts  = if_else(is_new_season, 0.5 * roll_ts + 0.5 * mean(ts, na.rm = TRUE), roll_ts)
      )
  } else {
    features <- features |>
      mutate(
        roll_pts_scored = if_else(is_new_season, 0.8 * roll_pts_scored + 0.2 * mean(team_score, na.rm
        roll_efg = if_else(is_new_season, 0.8 * roll_efg + 0.2 * mean(efg, na.rm = TRUE), roll_efg)
      )
  }
  features <- features |> ungroup()
}

features_selected <- features |>
  select(game_id, season, game_date, team_id, team_winner, team_home_away,
         roll_pts_scored, roll_oreb, roll_dreb, roll_ast, roll_stl, roll_blk,
         roll_tov, roll_efg, roll_tovp, roll_ftr, roll_ts, days_rest)

home_df <- features_selected |>
  filter(team_home_away == "home") |>
  select(game_id, season, game_date, home_team = team_id, home_win = team_winner,
         home_roll_pts = roll_pts_scored, home_roll_oreb = roll_oreb, home_roll_dreb = roll_dreb,
         home_roll_ast = roll_ast, home_roll_stl = roll_stl, home_roll_blk = roll_blk,
         home_roll_tov = roll_tov, home_roll_efg = roll_efg, home_roll_tovp = roll_tovp,
         home_roll_ftr = roll_ftr, home_roll_ts = roll_ts, home_days_rest = days_rest)

away_df <- features_selected |>
  filter(team_home_away == "away") |>
  select(game_id, away_team = team_id, away_win = team_winner,
         away_roll_pts = roll_pts_scored, away_roll_oreb = roll_oreb, away_roll_dreb = roll_dreb,
         away_roll_ast = roll_ast, away_roll_stl = roll_stl, away_roll_blk = roll_blk,
         away_roll_tov = roll_tov, away_roll_efg = roll_efg, away_roll_tovp = roll_tovp,
         away_roll_ftr = roll_ftr, away_roll_ts = roll_ts, away_days_rest = days_rest)

model_df <- inner_join(home_df, away_df, by = "game_id") |>
  mutate(
    target_home_win = factor(if_else(home_win == TRUE, "Win", "Loss"), levels = c("Win", "Loss")),
    diff_roll_pts   = home_roll_pts - away_roll_pts,
    diff_roll_oreb  = home_roll_oreb - away_roll_oreb,
    diff_roll_dreb  = home_roll_dreb - away_roll_dreb,
    diff_roll_treb  = (home_roll_oreb + home_roll_dreb) - (away_roll_oreb + away_roll_dreb),
```

```
      diff_roll_ast   = home_roll_ast - away_roll_ast,
      diff_roll_stl   = home_roll_stl - away_roll_stl,
      diff_roll_blk   = home_roll_blk - away_roll_blk,
      diff_roll_tov   = home_roll_tov - away_roll_tov,
      diff_roll_efg   = home_roll_efg - away_roll_efg,
      diff_roll_tovp  = home_roll_tovp - away_roll_tovp,
      diff_roll_ftr   = home_roll_ftr - away_roll_ftr,
      diff_roll_ts    = home_roll_ts - away_roll_ts,
      diff_days_rest  = home_days_rest - away_days_rest
    ) |>
    drop_na()

  return(model_df)
}

# Evaluate the 3 Offseason Methods out-of-sample on 2026 season
eval_offseason_method <- function(method_name) {
  df <- build_feature_df(box_scores, window_size = n_rolling_days, method = method_name)
  train_data <- df |> filter(season < 2026)
  test_data  <- df |> filter(season >= 2026)

  rec <- recipe(target_home_win ~ ., data = train_data) |>
    step_rm(
      game_id, game_date, season, home_team, away_team, home_win, away_win,
      home_roll_ts, home_roll_ftr, home_roll_efg, home_roll_pts, home_roll_tovp,
      home_roll_oreb, home_roll_dreb, home_roll_ast, home_roll_stl, home_roll_blk, home_roll_tov,
      away_roll_ts, away_roll_ftr, away_roll_efg, away_roll_pts, away_roll_tovp,
      away_roll_oreb, away_roll_dreb, away_roll_ast, away_roll_stl, away_roll_blk, away_roll_tov,
      home_days_rest, away_days_rest
    ) |>
    step_normalize(all_numeric_predictors())

  rf <- rand_forest(trees = 200, mode = "classification") |> set_engine("ranger", num.threads = 4)
  wf <- workflow() |> add_recipe(rec) |> add_model(rf)
  fit_m <- fit(wf, train_data)

  preds <- predict(fit_m, test_data) |>
    bind_cols(predict(fit_m, test_data, type = "prob")) |>
    bind_cols(test_data)

  acc <- preds |> accuracy(truth = target_home_win, estimate = .pred_class) |> pull(.estimate)
  auc <- preds |> roc_auc(truth = target_home_win, .pred_Win, event_level = "first") |> pull(.estimate)
  brier <- mean((if_else(preds$target_home_win == "Win", 1, 0) - preds$.pred_Win)^2)

  tibble(Method = method_name, Accuracy = round(acc, 4), `ROC-AUC` = round(auc, 4), `Brier Score` = rou
}

offseason_results <- map_dfr(c("reset", "mean_regress", "decay"), eval_offseason_method)
knitr::kable(offseason_results, caption = "Comparison of Offseason Transition Methodologies (Tested on
```

Table 1: Comparison of Offseason Transition Methodologies (Tested on 2026 Season)

Method	Accuracy	ROC-AUC	Brier Score
reset	0.6333	0.6665	0.2264
mean_regress	0.6202	0.6643	0.2265
decay	0.6202	0.6600	0.2278

Model Benchmarking Architecture

We construct the primary modeling dataset using Method A (Fresh Reset) and fit three candidate algorithms on historical seasons (2019–2025): 1. **Logistic Regression (Baseline)** 2. **Random Forest (`ranger`)** 3. **Gradient Boosted Trees (`xgboost`)**

```
model_df <- build_feature_df(box_scores, window_size = n_rolling_days, method = "reset")

train_data <- model_df |> filter(season < 2026)
test_data  <- model_df |> filter(season >= 2026)

# Recipe without data leakage (season explicitly removed, raw home/away components stripped)
nba_recipe <- recipe(target_home_win ~ ., data = train_data) |>
  step_rm(
    game_id, game_date, season, home_team, away_team, home_win, away_win,
    home_roll_ts, home_roll_ftr, home_roll_efg, home_roll_pts, home_roll_tovp,
    home_roll_oreb, home_roll_dreb, home_roll_ast, home_roll_stl, home_roll_blk, home_roll_tov,
    away_roll_ts, away_roll_ftr, away_roll_efg, away_roll_pts, away_roll_tovp,
    away_roll_oreb, away_roll_dreb, away_roll_ast, away_roll_stl, away_roll_blk, away_roll_tov,
    home_days_rest, away_days_rest
  ) |>
  step_normalize(all_numeric_predictors())

# 1. Logistic Regression
lr_spec <- logistic_reg() |> set_engine("glm")
lr_wf <- workflow() |> add_recipe(nba_recipe) |> add_model(lr_spec)
lr_fit <- fit(lr_wf, train_data)

# 2. Random Forest
rf_spec <- rand_forest(trees = 250, mode = "classification") |> set_engine("ranger", importance = "impu
rf_wf <- workflow() |> add_recipe(nba_recipe) |> add_model(rf_spec)
rf_fit <- fit(rf_wf, train_data)

# 3. XGBoost
xgb_spec <- boost_tree(trees = 150, tree_depth = 3, mode = "classification") |> set_engine("xgboost", n
xgb_wf <- workflow() |> add_recipe(nba_recipe) |> add_model(xgb_spec)
xgb_fit <- fit(xgb_wf, train_data)

# Evaluate all 3 models on 2026 test set
eval_model <- function(fitted_wf, model_name) {
  preds <- predict(fitted_wf, test_data) |>
    bind_cols(predict(fitted_wf, test_data, type = "prob")) |>
    bind_cols(test_data)

  acc <- preds |> accuracy(truth = target_home_win, estimate = .pred_class) |> pull(.estimate)
```

```
  auc <- preds |> roc_auc(truth = target_home_win, .pred_Win, event_level = "first") |> pull(.estimate)
  brier <- mean((if_else(preds$target_home_win == "Win", 1, 0) - preds$.pred_Win)^2)
  logloss <- mn_log_loss(preds, truth = target_home_win, .pred_Win, event_level = "first") |> pull(.esti

  tibble(
    Model = model_name,
    Accuracy = round(acc, 4),
    `ROC-AUC` = round(auc, 4),
    `Brier Score` = round(brier, 4),
    `Log-Loss` = round(logloss, 4)
  )
}

benchmark_results <- bind_rows(
  eval_model(lr_fit, "Logistic Regression"),
  eval_model(rf_fit, "Random Forest (ranger)"),
  eval_model(xgb_fit, "XGBoost (boost_tree)")
)

knitr::kable(benchmark_results, caption = "Model Architecture Benchmark Performance on 2026 Test Season
```

Table 2: Model Architecture Benchmark Performance on 2026 Test Season

Model	Accuracy	ROC-AUC	Brier Score	Log-Loss
Logistic Regression	0.6496	0.7012	0.2178	0.6267
Random Forest (ranger)	0.6333	0.6685	0.2257	0.6432
XGBoost (boost_tree)	0.6248	0.6519	0.2335	0.6643

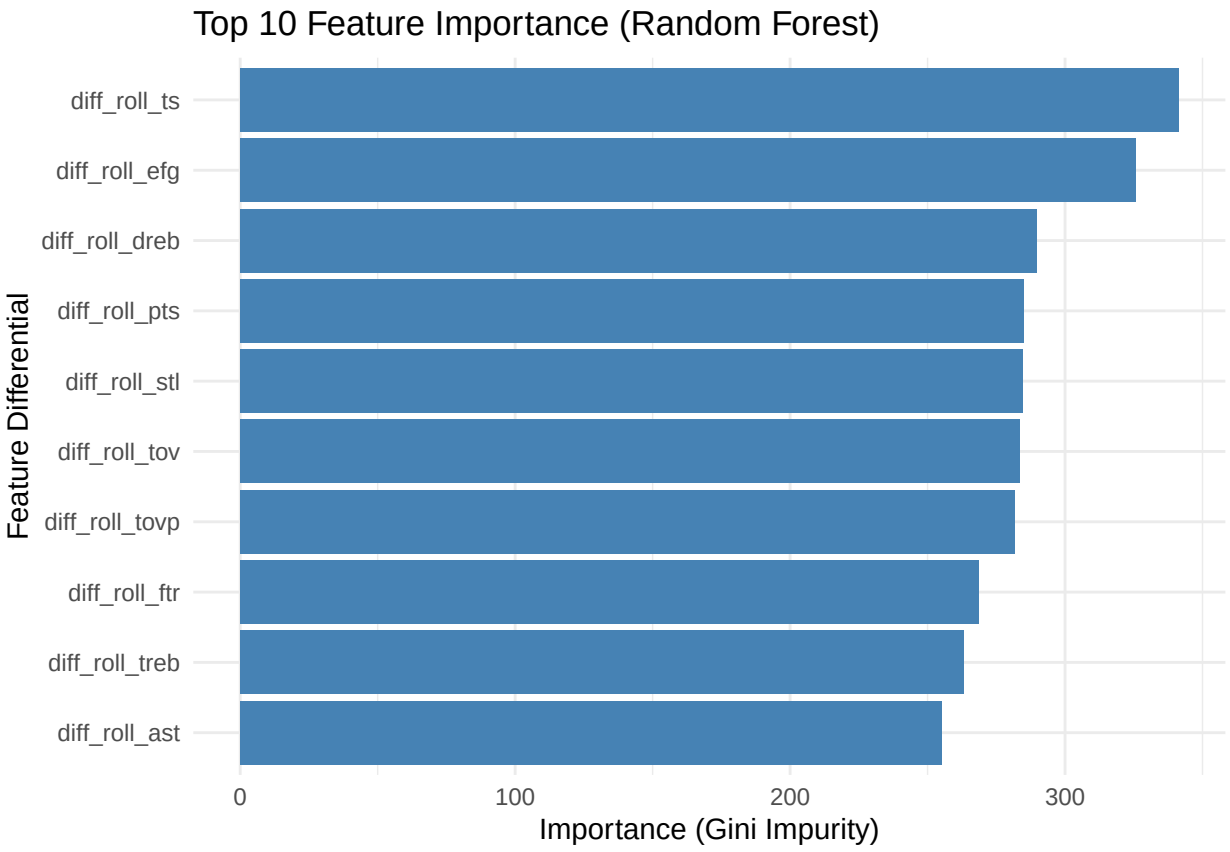
Feature Importance

We inspect feature importance for the Random Forest model to determine key drivers of game outcome predictions:

```
rf_engine <- extract_fit_engine(rf_fit)
imp_vec <- rf_engine$variable.importance

importance_df <- data.frame(
  feature = names(imp_vec),
  importance = as.numeric(imp_vec)
) |>
  arrange(desc(importance)) |>
  head(10)

ggplot(importance_df, aes(x = reorder(feature, importance), y = importance)) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  theme_minimal() +
  labs(
    title = "Top 10 Feature Importance (Random Forest)",
    x = "Feature Differential",
    y = "Importance (Gini Impurity)"
  )
```



2027 Season Schedule & Dynamic Simulation

We construct an 82-game regular season schedule per team (1,230 games total across 30 NBA teams) and perform a 10,000-season Monte Carlo simulation using model probabilities from the best performing architecture (`rf_fit`).

```
team_lookup <- box_scores |>
  select(team_id, team_name = team_short_display_name) |>
  distinct() |>
  mutate(team_id = as.integer(team_id))

# Latest 2026 Team Profiles
team_profiles_2026 <- model_df |>
  filter(season == 2026) |>
  group_by(home_team) |>
  slice_tail(n = 1) |>
  ungroup()

teams <- unique(team_lookup$team_id)
set.seed(42)

# Generate balanced 82-game regular season schedule matrix (41 home, 41 away per team)
n_teams <- length(teams)
sched_list <- list()

for (i in 1:n_teams) {
```

```
  t_home <- teams[i]
  others <- setdiff(teams, t_home)
  # Repeat opponents to reach 41 home games
  opps <- sample(rep(others, length.out = 41))
  sched_list[[i]] <- tibble(
    home_team = t_home,
    away_team = opps,
    game_date = Sys.Date() + 1:41
  )
}

sched_2027 <- bind_rows(sched_list) |>
  mutate(
    game_id = row_number(),
    season = 2027,
    home_win = FALSE,
    away_win = FALSE,
    home_days_rest = 2,
    away_days_rest = 2,
    diff_days_rest = 0
  )

# Extract baseline team profile vectors
profiles <- team_profiles_2026 |>
  select(team_id = home_team, starts_with("home_roll_"))

names(profiles) <- gsub("^home_roll_", "roll_", names(profiles))

sched_2027_features <- sched_2027 |>
  left_join(profiles, by = c("home_team" = "team_id")) |>
  rename_with(~ paste0("home_", .), starts_with("roll_")) |>
  left_join(profiles, by = c("away_team" = "team_id")) |>
  rename_with(~ paste0("away_", .), starts_with("roll_")) |>
  mutate(
    diff_roll_pts   = home_roll_pts - away_roll_pts,
    diff_roll_oreb  = home_roll_oreb - away_roll_oreb,
    diff_roll_dreb  = home_roll_dreb - away_roll_dreb,
    diff_roll_treb  = (home_roll_oreb + home_roll_dreb) - (away_roll_oreb + away_roll_dreb),
    diff_roll_ast   = home_roll_ast - away_roll_ast,
    diff_roll_stl   = home_roll_stl - away_roll_stl,
    diff_roll_blk   = home_roll_blk - away_roll_blk,
    diff_roll_tov   = home_roll_tov - away_roll_tov,
    diff_roll_efg   = home_roll_efg - away_roll_efg,
    diff_roll_tovp  = home_roll_tovp - away_roll_tovp,
    diff_roll_ftr   = home_roll_ftr - away_roll_ftr,
    diff_roll_ts    = home_roll_ts - away_roll_ts,
    diff_days_rest  = 0,
    target_home_win = factor("Win", levels = c("Win", "Loss"))
  )

# Predict win probabilities for each scheduled game
predictions_2027 <- sched_2027_features |>
  bind_cols(predict(rf_fit, sched_2027_features, type = "prob"))
```

```r
# Analytical 95% Confidence Intervals (Poisson-Binomial)
home_var <- predictions_2027 |>
  group_by(team_id = home_team) |>
  summarise(home_wins = sum(.pred_Win), home_var = sum(.pred_Win * (1 - .pred_Win)))

away_var <- predictions_2027 |>
  group_by(team_id = away_team) |>
  summarise(away_wins = sum(.pred_Loss), away_var = sum(.pred_Loss * (1 - .pred_Loss)))

analytical_ci <- home_var |>
  left_join(away_var, by = "team_id") |>
  left_join(team_lookup, by = "team_id") |>
  mutate(
    Projected_Wins = round(home_wins + away_wins, 1),
    Projected_Losses = round(82 - Projected_Wins, 1),
    SE = round(sqrt(home_var + away_var), 2),
    CI_Lower_95 = round(Projected_Wins - 1.96 * SE, 1),
    CI_Upper_95 = round(Projected_Wins + 1.96 * SE, 1)
  ) |>
  arrange(desc(Projected_Wins)) |>
  mutate(Rank = row_number()) |>
  select(Rank, Team = team_name, Projected_Wins, Projected_Losses, SE, CI_Lower_95, CI_Upper_95)

knitr::kable(head(analytical_ci, 10), caption = "Top 10 Projected 2027 NBA Standings with Analytical 95
```

Table 3: Top 10 Projected 2027 NBA Standings with Analytical 95% CIs

Rank	Team	Projected_Wins	Projected_Losses	SE	CI_Lower_95	CI_Upper_95
1	Knicks	52.6	29.4	4.25	44.3	60.9
2	Thunder	49.3	32.7	4.34	40.8	57.8
3	Clippers	48.7	33.3	4.30	40.3	57.1
4	Celtics	48.4	33.6	4.39	39.8	57.0
5	Nuggets	48.2	33.8	4.39	39.6	56.8
6	Rockets	46.6	35.4	4.41	38.0	55.2
7	Spurs	46.2	35.8	4.42	37.5	54.9
8	Raptors	45.6	36.4	4.39	37.0	54.2
9	Hornets	44.6	37.4	4.44	35.9	53.3
10	Heat	44.3	37.7	4.43	35.6	53.0

Monte Carlo Simulation (10,000 Seasons)

```r
n_sims <- 10000
n_games <- nrow(predictions_2027)

# Fast vectorized matrix simulation (1,230 games x 10,000 simulated seasons)
sim_matrix <- matrix(rbinom(n_games * n_sims, 1, predictions_2027$.pred_Win), nrow = n_games, ncol = n_s

home_teams_vec <- predictions_2027$home_team
away_teams_vec <- predictions_2027$away_team

teams_vec <- unique(team_lookup$team_id)
```

```
team_wins_matrix <- sapply(teams_vec, function(t_id) {
  colSums(sim_matrix[home_teams_vec == t_id, , drop = FALSE]) +
  colSums((1 - sim_matrix)[away_teams_vec == t_id, , drop = FALSE])
})

sim_results <- tibble(
  sim = rep(1:n_sims, times = length(teams_vec)),
  team_id = rep(teams_vec, each = n_sims),
  total_wins = as.vector(team_wins_matrix)
)

standings_sim_summary <- sim_results |>
  left_join(team_lookup, by = "team_id") |>
  group_by(Team = team_name) |>
  summarise(
    Mean_Wins = round(mean(total_wins), 1),
    Median_Wins = median(total_wins),
    `5th_Pct` = quantile(total_wins, 0.05),
    `95th_Pct` = quantile(total_wins, 0.95),
    `2.5th_Pct` = quantile(total_wins, 0.025),
    `97.5th_Pct` = quantile(total_wins, 0.975)
  ) |>
  arrange(desc(Mean_Wins)) |>
  mutate(Rank = row_number()) |>
  select(Rank, Team, Mean_Wins, Median_Wins, `5th_Pct`, `95th_Pct`, `2.5th_Pct`, `97.5th_Pct`)

knitr::kable(head(standings_sim_summary, 10), caption = "Top 10 Projected 2027 NBA Standings from 10,000
```

Table 4: Top 10 Projected 2027 NBA Standings from 10,000 Monte Carlo Simulations

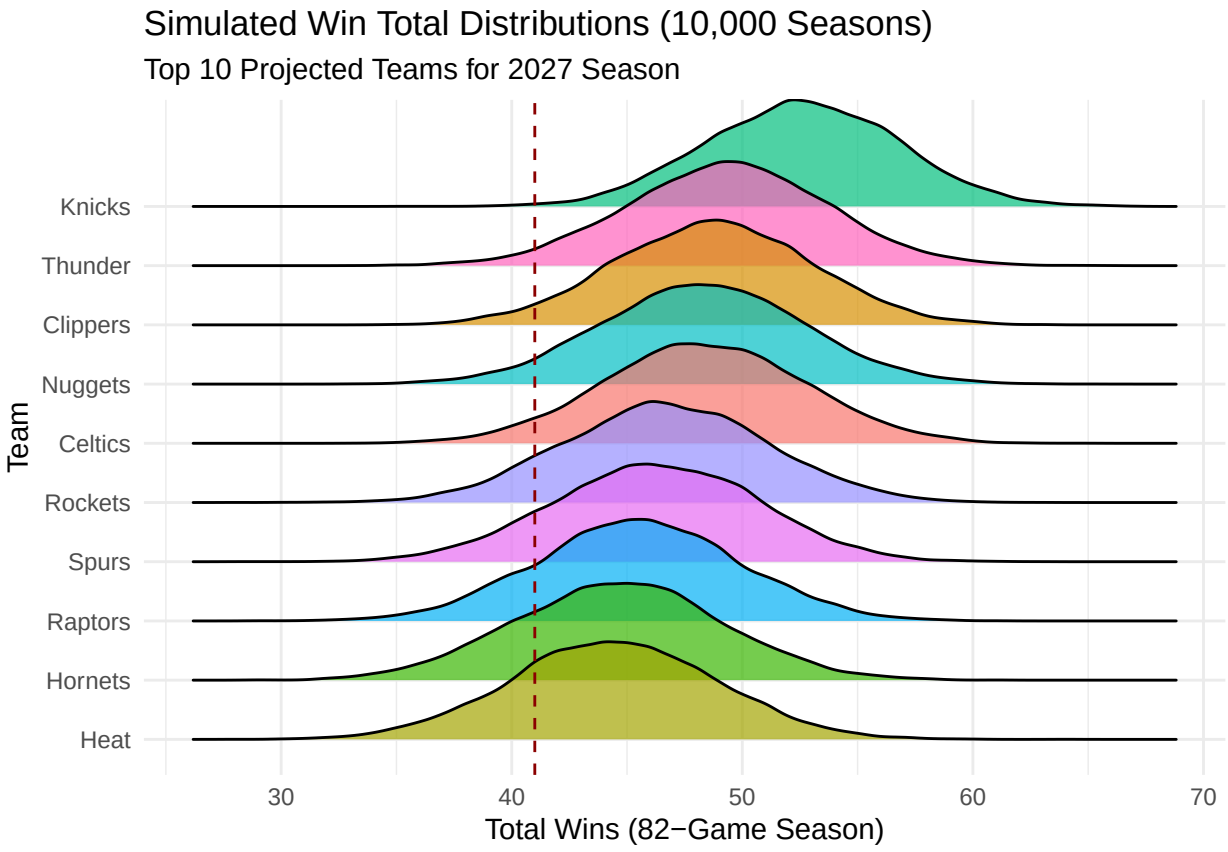
Rank	Team	Mean_Wins	Median_Wins	5th_Pct	95th_Pct	2.5th_Pct	97.5th_Pct
1	Knicks	52.6	53	46	59	44	61
2	Thunder	49.3	49	42	56	41	58
3	Clippers	48.7	49	42	56	40	57
4	Celtics	48.3	48	41	55	40	57
5	Nuggets	48.3	48	41	55	40	57
6	Rockets	46.7	47	39	54	38	55
7	Spurs	46.2	46	39	53	37	55
8	Raptors	45.5	45	38	53	37	54
9	Hornets	44.6	45	37	52	36	53
10	Heat	44.4	44	37	52	36	53

Win Total Density Ridge Plot

```
top10_teams <- head(standings_sim_summary$Team, 10)

sim_results |>
  left_join(team_lookup, by = "team_id") |>
  filter(team_name %in% top10_teams) |>
  ggplot(aes(x = total_wins, y = reorder(team_name, total_wins, median), fill = team_name)) +
  geom_density_ridges(alpha = 0.7, show.legend = FALSE) +
```

```
  geom_vline(xintercept = 41, linetype = "dashed", color = "darkred") +
  theme_minimal() +
  labs(
    title = "Simulated Win Total Distributions (10,000 Seasons)",
    subtitle = "Top 10 Projected Teams for 2027 Season",
    x = "Total Wins (82-Game Season)",
    y = "Team"
  )
```



Conclusion & Next Steps

This refactored NBA analytics pipeline successfully resolves historical data leakage bugs, benchmarks candidate machine learning algorithms, and simulates full 82-game season standings.

For future enhancements (including player-level RAPM integration, injury report tracking, and line-ups synergy), refer to the project Wishlist Roadmap.